

DESIGN-PATTERN

Cấu trúc dữ liệu của YouTube, GitHub đã giúp tôi 'flex' trong buổi phỏng vấn Senior | Bloom Filters

Giới thiệu và các bài toán thực tế cần Bloom Filter

Chào mừng bạn quay trở lại với series về JavaScript (hoặc các chủ đề lập trình liên quan). Hôm nay chúng ta sẽ nói về một cấu trúc dữ liệu rất hữu ích trong các hệ thống lớn: Bloom Filter.

Bạn đã bao giờ gặp phải những tình huống sau chưa?

- Đầu tiên: Khi xây dựng chức năng đăng ký tài khoản cho một nền tảng như Facebook, Instagram hay X (Twitter), với hàng tỷ người dùng. Làm sao để nhanh chóng kiểm tra xem một email đã được đăng ký hay chưa, mà không cần quét toàn bộ cơ sở dữ liệu khổng lồ?
 - Thứ hai: Trong hệ thống như Netflix hoặc YouTube, khi phân phối quảng cáo. Mỗi quảng cáo chỉ nên được hiển thị cho một người dùng đúng một lần. Làm sao biết chắc chắn rằng quảng cáo X đã được gửi đến user Y hay chưa, trong hàng tỷ người dùng, mà không cho phép trùng lặp?
 - Thứ ba: Trong kiến trúc cache (bộ nhớ đệm). Khi người dùng yêu cầu một bài viết theo ID, nếu ID đó có trong cache (ví dụ Redis), ta trả dữ liệu ngay lập tức mà không cần hỏi database. Nhưng nếu ai đó cố tình truyền ID âm, ID không hợp lệ hoặc định dạng lạ để vượt qua validation, cache sẽ miss và phải hỏi database. Làm sao biết nhanh chóng ID đó chắc chắn không tồn tại trong database để trả về lỗi ngay, tránh truy vấn vô ích?
- Đây chính là những bài toán mà Bloom Filter được thiết kế để giải quyết một cách tối ưu, đặc biệt khi cần tốc độ cao và tiết kiệm bộ nhớ.

Bloom Filter là gì và ba loại câu trả lời của nó

Bloom Filter là một cấu trúc dữ liệu xác suất (probabilistic), giúp kiểm tra xem một phần tử có thể tồn tại hay chắc chắn không tồn tại trong một tập hợp lớn.

Nó có ba loại câu trả lời:

- No (Âm tính thật - True Negative): "Không có" → Tin tưởng 100%. Phần tử chắc chắn không tồn tại trong tập hợp. Đây là trường hợp lý tưởng cho các bài toán như cho phép đăng ký email mới, hoặc từ chối ID không tồn tại.
 - Yes - True Positive (Dương tính thật): "Có" và thực tế đúng là có → Phần tử thực sự tồn tại.
 - Yes - False Positive (Dương tính giả): "Có" nhưng thực tế không có → Đây là lỗi duy nhất mà Bloom Filter có thể mắc phải, gọi là dương tính giả.
- Bloom Filter không bao giờ trả về False Negative (nói "không có" trong khi thực tế có). Nó chỉ có thể nhầm theo hướng "có thể có" khi thực tế không có.

Ví dụ minh họa đơn giản với một hàm băm

Giả sử chúng ta muốn kiểm tra chuỗi "tip JavaScript" có trong tập hợp hay không.

Bloom Filter sử dụng một mảng bit (bit array) có kích thước cố định, ví dụ $m = 10$ bit (tất cả ban đầu là 0).

Chúng ta dùng hàm băm (hash function) để chuyển chuỗi thành một số, rồi lấy modulo với kích thước m để ra chỉ số trong mảng bit.

Ví dụ: $\text{hash}(\text{"tip JavaScript"}) = 125 \rightarrow 125 \% 10 = 5 \rightarrow$ Đặt bit thứ 5 thành 1.

Tương tự:

- $\text{hash}(\text{"tip go"}) = 290 \rightarrow 290 \% 10 = 0 \rightarrow$ Đặt bit 0 = 1

- $\text{hash}(\text{"tip Java"}) =$ giá trị nào đó cũng ra 5 $\rightarrow$ bit 5 vẫn là 1 (trùng không sao).

Khi kiểm tra một phần tử, ta băm nó ra chỉ số và xem bit tương ứng có phải 1 không. Nếu tất cả bit cần kiểm tra đều là 1 $\rightarrow$ kết luận "có thể có" (Yes). Nếu chỉ một bit là 0 $\rightarrow$ chắc chắn "không có" (No).

False Positive xảy ra như thế nào?

Giả sử ta kiểm tra "tip note", hash ra 165 $\rightarrow 165 \% 10 = 5$. Bit thứ 5 đã được đặt bởi "tip JavaScript" trước đó.

$\rightarrow$ Bloom Filter trả về Yes (có thể có), nhưng thực tế "tip note" chưa từng được thêm vào. Đây chính là False Positive do xung đột băm (hash collision).

Cách giảm False Positive: Sử dụng nhiều hàm băm

Trong thực tế, Bloom Filter không dùng chỉ một hàm băm (vì dễ collision, đặc biệt khi mảng bit nhỏ).

Thay vào đó, dùng k hàm băm khác nhau (thường $k \geq 3$).

Khi thêm phần tử: Băm k lần $\rightarrow$ đặt k bit tương ứng thành 1.

Khi kiểm tra: Băm k lần $\rightarrow$ nếu tất cả k bit đều là 1 $\rightarrow$ Yes (có thể có). Nếu chỉ một bit là 0 $\rightarrow$ No (chắc chắn không có).

Việc dùng nhiều hàm băm độc lập làm giảm đáng kể xác suất collision đồng thời ở tất cả k vị trí $\rightarrow$ giảm tỷ lệ False Positive.

Nếu vẫn còn False Positive (hiếm), ta có thể tăng k (số hàm băm) hoặc tăng kích thước mảng bit m . Tuy nhiên, Bloom Filter chấp nhận đánh đổi: tiết kiệm bộ nhớ cực lớn (chỉ lưu bit 0/1 thay vì toàn bộ chuỗi), tốc độ rất nhanh (chỉ tính hash và check bit), nhưng chấp nhận một tỷ lệ nhỏ dương tính giả.

Ứng dụng thực tế và Redis Bloom Filter

Bloom Filter được dùng rộng rãi trong các hệ thống lớn để giảm tải database/cache, chống cache penetration (truy vấn vô nghĩa), kiểm tra duplicate quảng cáo, lọc email đã đăng ký, v.v.

Trong Redis, có module RedisBloom (hoặc Redis Stack) hỗ trợ Bloom Filter sẵn, với các lệnh như BF.RESERVE (tạo filter), BF.ADD (thêm phần tử), BF.EXISTS (kiểm tra tồn tại).

Đây là một trong những cấu trúc dữ liệu probabilistic phổ biến nhất mà các kỹ sư hệ thống lớn thường sử dụng.

SOLID: Open Closes Principle OCP - Mức độ khó: 3, Mức độ phổ biến: 5

Giới thiệu về nguyên tắc Open-Closed Principle (OCP) trong SOLID

Chào mừng bạn quay trở lại với series về Design Pattern và SOLID. Hôm nay chúng ta sẽ tìm hiểu nguyên tắc O trong SOLID: Open-Closed Principle (Nguyên tắc Đóng-Mở), thường được gọi tắt là OCP.

Nguyên tắc này được coi là một trong những nguyên tắc quan trọng nhất của lập trình hướng đối tượng. Theo định nghĩa kinh điển của Robert C. Martin (Uncle Bob):

"Các thực thể phần mềm (class, module, hàm, v.v.) nên mở để mở rộng (open for extension), nhưng đóng để sửa đổi (closed for modification)."

Nói đơn giản:

- Mở để mở rộng: Bạn có thể thêm chức năng mới mà không cần chạm vào code cũ.
- Đóng để sửa đổi: Khi cần thêm tính năng, bạn không sửa trực tiếp code hiện có (để tránh làm hỏng những phần đang chạy ổn định).

Nhiều người mới học ban đầu hay bối rối giữa "sửa đổi" và "mở rộng". Qua thực tế làm dự án (đặc biệt với các dự án từ Nhật Bản), ta nhận ra: sửa đổi là thay đổi code cũ → rủi ro cao, dễ gây bug. Mở rộng là thêm code mới (thông qua kế thừa, interface, composition) → an toàn và dễ bảo trì hơn.

Ví dụ thực tế: Cái ổ cắm điện

Hãy tưởng tượng bạn mua một cái ổ cắm điện có 2 lỗ cắm. Ban đầu, bạn chỉ cắm TV và tủ lạnh — mọi thứ hoàn hảo.

Nhưng một ngày trời nóng cực kỳ (El Niño chẳng hạn), bạn phải mua thêm quạt máy để làm mát. Bây giờ ổ cắm chỉ có 2 lỗ, mà bạn cần 3 thiết bị. Bạn sẽ làm gì?

- Cách 1 (không tốt): Mua ổ cắm mới hoàn toàn khác → tương đương thay đổi toàn bộ hệ thống cũ.
- Cách 2 (cũng không tốt): Sửa ổ cắm cũ bằng cách khoan thêm lỗ → rủi ro hỏng ổ cũ, không an toàn.

Trong lập trình, chúng ta cũng gặp tình huống tương tự: hệ thống đang chạy tốt với vài tính năng, nhưng khách hàng yêu cầu thêm tính năng mới → nếu sửa code cũ thì nguy cơ cao gây lỗi.

Lập trình viên thứ nhất: Không tuân thủ OCP (cách sai)

Giả sử chúng ta có class Socket (ổ cắm) quản lý việc kết nối các thiết bị.

Ban đầu:

- Class TV có method connect() in ra "TV connected".
- Class Fridge (tủ lạnh) cũng có connect().

- Trong Socket: constructor nhận TV và Fridge, method activate() gọi connect() của cả hai.
Code chạy tốt với 2 thiết bị.

Khi thêm Fan (quạt):

- Tạo class Fan với connect().
- Sửa constructor của Socket để nhận thêm Fan.
- Sửa method activate() để gọi thêm fan.connect().

Kết quả: Mỗi lần thêm thiết bị mới (máy sưởi, máy giặt, đèn, v.v.), bạn phải sửa trực tiếp class Socket → vi phạm OCP.

Hậu quả:

- Class Socket phụ thuộc chặt vào các class cụ thể (TV, Fridge, Fan...).
- Code khó bảo trì, dễ lỗi khi sửa.
- Mỗi thay đổi đều phải test lại toàn bộ → tốn kém và rủi ro cao.

Lập trình viên thứ hai: Tuân thủ OCP (cách đúng)

Lập trình viên này áp dụng OCP bằng cách làm cho Socket không phụ thuộc vào các thiết bị cụ thể, mà chỉ làm việc với một danh sách thiết bị chung.

Cách triển khai:

- Tạo một mảng (array) devices ban đầu rỗng.
- Thêm method addDevice(device) để push thiết bị vào mảng.

Method activate() chỉ đơn giản duyệt qua mảng và gọi device.connect() cho từng thiết bị (không cần biết thiết bị là gì).

Cách sử dụng:

```
const socket = new Socket();
socket.addDevice(new TV());
socket.addDevice(new Fridge());
socket.activate(); // Kết nối TV và Fridge
// Sau này thêm quạt
socket.addDevice(new Fan());
socket.activate(); // Tự động kết nối thêm quạt, không sửa code Socket
```

Kết quả:

- Class Socket đóng cho sửa đổi (không cần sửa constructor hay activate khi thêm thiết bị).
- Mở cho mở rộng (chỉ cần tạo class mới như Fan, Lamp, rồi add vào).
- Code linh hoạt, dễ bảo trì, giảm rủi ro bug khi mở rộng hệ thống.

Lợi ích của việc tuân thủ OCP

- Hệ thống phần mềm luôn thay đổi và phát triển liên tục.
- Nếu thiết kế tốt từ đầu (theo OCP), bạn có thể thêm tính năng mới mà không làm hỏng code cũ.
- Giảm lỗi, dễ test, dễ bảo trì lâu dài.

- Đặc biệt hữu ích trong các dự án lớn, nơi yêu cầu thay đổi thường xuyên.
Nguyên tắc này không phải lúc nào cũng áp dụng cứng nhắc, nhưng khi làm việc với hệ thống có khả năng mở rộng cao, OCP giúp code "sống lâu" hơn rất nhiều.

DESIGN PATTERN vs SOLID: Sự khác nhau là gì? Nên học gì trước? Tôi là anti của S trong SOLID.

Giới thiệu: Design Pattern vs SOLID – Sự khác biệt là gì? Nên học gì trước?

Chào mừng bạn quay trở lại với series về JavaScript và các chủ đề lập trình nâng cao. Hôm nay chúng ta sẽ giải đáp một câu hỏi rất phổ biến từ cộng đồng: Design Pattern và SOLID khác nhau như thế nào? Tại sao lại học chúng? Và nên học cái nào trước?

Nhiều bạn thắc mắc tại sao series Design Pattern mới đi đến 14 pattern, trong khi Gang of Four (GoF) đã đề xuất 23 design pattern kinh điển trong cuốn sách nổi tiếng năm 1994.

Thực ra, 23 cái đó là bộ sưu tập các mẫu thiết kế hướng đối tượng cơ bản, được phân loại thành Creational, Structural và Behavioral. Chúng là những "mẫu sẵn" để giải quyết vấn đề phổ biến, giúp code tái sử dụng, dễ hiểu và dễ bảo trì hơn.

Design Pattern là gì?

Hãy nghĩ Design Pattern như những mẫu nhà hoặc chiêu thức võ thuật đã được kiểm chứng qua thời gian. Khi xây dựng ứng dụng, gặp vấn đề quen thuộc (ví dụ: tạo object mà không chỉ định class cụ thể, quản lý trạng thái, kết nối module mà không phụ thuộc chặt...), bạn chỉ cần "lấy mẫu" đó ra áp dụng là nhanh gọn.

Thậm chí bạn có thể quên tên pattern (như Strategy, Observer, Factory...), quên cấu trúc chi tiết, nhưng ý tưởng cốt lõi đằng sau nó vẫn giúp bạn giải quyết vấn đề hàng ngày.

Nhiều lập trình viên dùng pattern mà không biết tên – đó là dấu hiệu bạn đã "nội hóa" nó.

Lợi ích lớn nhất: Đứng trên vai những người không lờ (các ông cha lập trình trước), code sạch hơn, đồng đội dễ đọc code hơn, giảm lỗi và tăng độ tin cậy.

SOLID là gì? Và tại sao nên học SOLID trước Design Pattern?

SOLID là bộ 5 nguyên tắc cơ bản của lập trình hướng đối tượng (OOP), do Robert C. Martin (Uncle Bob) tổng hợp:

- S: Single Responsibility Principle (Trách nhiệm duy nhất)
- O: Open-Closed Principle
- L: Liskov Substitution Principle
- I: Interface Segregation Principle
- D: Dependency Inversion Principle

Nếu ví Design Pattern như chiêu thức võ thuật (các đòn đánh cụ thể của 12 môn phái), thì SOLID chính là nội công – nền tảng sức mạnh bên trong. Rèn nội công tốt trước, sau đó học chiêu thức sẽ đạt hiệu quả gấp đôi với một nửa công sức.

Vì vậy, lý tưởng là học SOLID trước (hoặc song song) để hiểu "tại sao" pattern tồn tại và cách áp dụng chúng đúng cách. SOLID giúp xây dựng tư duy backend/kiến trúc vững chắc, áp dụng được cho mọi ngôn ngữ (Node.js, Java, Go, PHP...).

Quan điểm cá nhân: Tôi là "anti-fan" của chữ S – Single Responsibility Principle (SRP)
Bây giờ đi sâu vào nguyên tắc S – Single Responsibility Principle (SRP): "Một class chỉ nên có một lý do để thay đổi" hoặc "một class chỉ chịu trách nhiệm một việc".

Định nghĩa nghe hay, nhưng theo kinh nghiệm thực tế, tôi thấy nó quá cứng nhắc và thường gây phiền phức trong lập trình hàng ngày. Nhiều lập trình viên có kinh nghiệm (kể cả chưa từng học SOLID) đã tự giác tách trách nhiệm rõ ràng theo bản năng – không cần nguyên tắc này cũng làm được.

Vấn đề gốc: Nếu một class làm 2 việc (P1 và P2), khi sửa P1 có thể làm hỏng P2 đang chạy tốt → rủi ro cao.

Giải pháp "chuẩn SOLID": Tách thành 2 class riêng (T1 làm P1, T2 làm P2) → sửa cái này không ảnh hưởng cái kia.

Nhưng thực tế: Nếu cứ tách nhỏ quá mức, code sẽ dư thừa class, file nhiều hơn, khó theo dõi, chi phí bảo trì tăng (file bloat). Nhiều dự án lớn vẫn vi phạm SRP một cách có chủ đích mà vẫn ổn.

Ví dụ thực tế: Hệ thống mua hàng bằng vàng (theo SRP vs cách thực tế)

Giả sử ta có class Item với method buyByGold(itemName) – in ra "Mua [item] bằng vàng".

Class Client dùng nó để mua: nhà, ô tô, MacBook, cổ phiếu... bằng vàng. Code đơn giản, chạy tốt.

Khách hàng phàn nàn: "Tôi có tiền mặt, sao chỉ mua bằng vàng được?" → Yêu cầu thêm mua bằng cash.

Cách theo SRP chuẩn:

- Tách thành BuyByGold và BuyByCash (2 class riêng).

- Sửa Client để dùng class phù hợp.

→ Mỗi lần thêm phương thức thanh toán mới (USD, VND, crypto...), phải tách class mới

→ code phình to, sửa nhiều nơi.

Cách thực tế của tôi (vi phạm SRP có chủ đích):

- Giữ nguyên class Access (hoặc PaymentProcessor).

- Thêm logic if-else hoặc switch: nếu item là MacBook thì mua bằng cash, nhà/ô tô bằng vàng...

→ Chỉ sửa một chỗ nhỏ, chi phí thấp, nhanh triển khai.

Nhưng rủi ro vẫn có: Khi khách yêu cầu phân biệt cash USD vs VND → logic if-else phình to, dễ lỗi (mua nhà bằng USD thay vì VND...). Lúc này phải refactor lại.

Kết luận: Khi nào vi phạm SRP là chấp nhận được?

- Nếu logic đơn giản, phạm vi thay đổi nhỏ → vi phạm SRP ở cấp code vẫn ổn, chi phí thấp hơn.
 - Khi phức tạp, rủi ro cao → nên tách trách nhiệm để giảm nguy cơ bug lan rộng.
- SRP không phải chân lý tuyệt đối. Nó tốt để tăng cohesion (độ gắn kết), nhưng áp dụng mù quáng dẫn đến over-engineering. Thực tế lập trình hàng ngày cân cân bằng: tuân thủ khi cần, linh hoạt khi phù hợp với ngữ cảnh dự án.
- Nguyên tắc này áp dụng không chỉ OOP mà còn functional/module programming.

Bridge Pattern: Có thể triển khai hệ thống thanh toán giống YOUTUBE | Khó:3, Phổ biến 3

Giới thiệu: Bridge Pattern – Mẫu thiết kế tách rời abstraction và implementation

Chào mừng bạn quay trở lại với series Design Pattern theo Gang of Four (GoF). Chúng ta đã đi qua Adapter Pattern (với ví dụ dùng adapter để chuyển Momo sang Visa để thanh toán hội viên YouTube). Hôm nay là Bridge Pattern – một trong những structural pattern quan trọng, thuộc nhóm 23 pattern kinh điển.

Bridge Pattern có độ khó khoảng 3 sao (trung bình khá), độ phổ biến cũng khoảng 3 sao (không quá phổ biến trong JavaScript thuần, nhưng rất hay trong Java hoặc các hệ thống lớn). Nếu bạn học Java thì không thể bỏ qua pattern này. Với JavaScript, nó ít được dùng trực tiếp hơn, nhưng hiểu rõ sẽ giúp tư duy thiết kế tốt hơn.

Bridge Pattern là gì?

Bridge Pattern giúp tách rời abstraction (giao diện trừu tượng) khỏi implementation (cách triển khai cụ thể), để cả hai có thể thay đổi độc lập mà không ảnh hưởng lẫn nhau.

Nói đơn giản:

- Abstraction: Phần "giao diện" mà client sử dụng (ví dụ: "thanh toán hội viên YouTube").
- Implementation: Phần "cách thức thực hiện" (thanh toán bằng Visa, Momo, PayPal, v.v.).

Bridge cho phép bạn thay đổi implementation (thêm phương thức thanh toán mới) mà không cần sửa abstraction, và ngược lại.

So sánh với Adapter Pattern (từ video trước)

1. Adapter: Dùng để làm cho hai giao diện không tương thích có thể "nói chuyện" được với nhau. Ví dụ: YouTube chỉ chấp nhận Visa, nhưng người dùng muốn dùng Momo → dùng Adapter để "chuyển đổi" Momo sang Visa (câu nói bắt buộc).

→ Adapter là "bắc cầu" khi hệ thống cũ không hỗ trợ.

2. Bridge: Không cần "chuyển đổi" gì cả. Hệ thống (YouTube) được thiết kế để hỗ trợ nhiều implementation độc lập ngay từ đầu.

→ YouTube mở ra hai cách thanh toán riêng biệt: dùng Momo thì thanh toán bằng Momo, dùng Visa thì thanh toán bằng Visa – không phụ thuộc lẫn nhau.

→ Bridge tách rời để mở rộng dễ dàng, tránh "phụ thuộc cứng".

Ví dụ thực tế: Hệ thống thanh toán hội viên YouTube

Ban đầu: YouTube chỉ chấp nhận Visa → muốn dùng Momo phải dùng Adapter để "giả lập" Visa.

Sau này (như thực tế): YouTube mở hỗ trợ Momo độc lập → giờ bạn có thể chọn:

- Thanh toán bằng Visa → dùng Visa trực tiếp.
- Thanh toán bằng Momo → dùng Momo trực tiếp.

Đây chính là Bridge: abstraction (đăng ký hội viên) tách khỏi implementation (cách thanh toán), hai bên thay đổi độc lập.

Cấu trúc code minh họa (JavaScript đơn giản)

1. Abstraction: Class trừu tượng hoặc interface cho việc thanh toán.

- PaymentProcessor: Có method pay(amount) – chỉ định "thanh toán số tiền X".

2. Implementation cụ thể: Các class triển khai riêng.

- VisaPaymentProcessor: Triển khai pay(amount) bằng thẻ Visa (cần cardNumber, expiry, cvv).

- MomoPaymentProcessor: Triển khai pay(amount) bằng ví Momo (cần phoneNumber).

3. Refined Abstraction: Class sử dụng abstraction để làm việc cao cấp hơn.

- YouTubeMembership: Có method registerWithPayment(amount, paymentProcessor) – nhận bất kỳ paymentProcessor nào (Visa hay Momo) và gọi paymentProcessor.pay(amount).

Cách sử dụng

// Visa

```
const visaProcessor = new VisaPaymentProcessor('1234-5678-9012-3456', '12/28', '123');
```

```
const membershipVisa = new YouTubeMembership();
```

```
membershipVisa.registerWithPayment(100, visaProcessor); // Thanh toán 100 USD bằng Visa
```

// Momo

```
const momoProcessor = new MomoPaymentProcessor('0123456789');
```

```
const membershipMomo = new YouTubeMembership();
```

```
membershipMomo.registerWithPayment(100, momoProcessor); // Thanh toán 100 USD bằng Momo
```

Kết quả:

- Bạn thêm phương thức thanh toán mới (PayPal, VNPay...) chỉ cần tạo class mới implement pay() → không sửa code YouTubeMembership.

- Bạn thay đổi logic đăng ký hội viên (thêm bước xác nhận email, v.v.) → không ảnh hưởng đến các payment processor.

Lợi ích của Bridge Pattern

- Giảm sự phụ thuộc chặt chẽ giữa abstraction và implementation.
- Dễ mở rộng: Thêm abstraction mới hoặc implementation mới mà không sửa code cũ.
- Tăng tính linh hoạt và tái sử dụng code.
- Đặc biệt hữu ích khi hệ thống có nhiều biến thể (như nhiều cổng thanh toán, nhiều nền tảng render đồ họa, nhiều driver database...).

Kết luận

Bridge Pattern giúp xây dựng hệ thống thanh toán giống YouTube hiện đại: hỗ trợ nhiều phương thức độc lập (Momo, Visa, v.v.) mà không cần "chuyển đổi" phức tạp như Adapter. Đây là pattern tốt để thiết kế hệ thống mở rộng từ đầu.

Adapter Pattern: module exports MoMo to Visa Card để đăng ký HỘI VIÊN trên YOUTUBE | Design Patterns

Giới thiệu: Adapter Pattern – Mẫu thiết kế phổ biến trong series Design Pattern

Chào mừng bạn quay trở lại với series Design Pattern theo Gang of Four (GoF). Đây là video thứ 10: Adapter Pattern – một trong những structural pattern quan trọng nhất.

Adapter có độ khó khoảng 2 sao (dễ học), nhưng độ phổ biến lên đến 4 sao (rất hay dùng trong dự án thực tế). Đây là pattern bạn nên nắm vững vì nó giúp giải quyết vấn đề giao diện không tương thích – một tình huống cực kỳ phổ biến trong lập trình.

Adapter Pattern là gì?

Adapter Pattern cho phép hai đối tượng có giao diện không tương thích vẫn có thể làm việc cùng nhau. Nó hoạt động như một "bộ chuyển đổi" hoặc "phiên dịch viên":

- Bên này có giao diện A (ví dụ: Visa).
- Bên kia có giao diện B (ví dụ: Momo).
- Adapter "dịch" B sang A để chúng giao tiếp được mà không cần thay đổi code gốc.

Ví dụ thực tế trong cuộc sống:

- Laptop dùng điện 12V, nhưng ổ cắm nhà là 220V → nếu cắm trực tiếp sẽ cháy nổ.
- Nhà sản xuất làm bộ adapter (sạc) chuyển đổi 220V xuống 12V → laptop dùng bình thường.

Adapter trong phần mềm cũng vậy: phổ biến không kém gì trong đời sống hàng ngày.

Câu chuyện thực tế: Đăng ký hội viên YouTube bằng Momo (khi chỉ hỗ trợ Visa)

Giả sử bạn có kênh YouTube (như kênh Tip JavaScript) và muốn tham gia hội viên để xem video độc quyền (ví dụ: khóa học backend e-commerce, Avantery, discount phức tạp với voucher, minimum order, maximum usage...).

Nhưng YouTube chỉ chấp nhận thanh toán bằng thẻ Visa (hoặc Mastercard). Bạn không có Visa, chỉ có Momo (ví điện tử phổ biến ở Việt Nam).

Bạn không muốn bỏ lỡ → giải pháp: dùng Adapter Pattern để "chuyển đổi" Momo thành Visa, giúp đăng ký hội viên thành công.

Luồng cơ bản:

- YouTube yêu cầu thanh toán Visa → trả về trạng thái success hoặc error.
- Nếu không có Visa → tạo Adapter "Momo to Visa":
 - Lấy thông tin Momo (số điện thoại, số dư...).
 - Adapter chuyển đổi và "giả lập" như thanh toán Visa (card number, expiry, CVV...).
 - Gọi API Visa của YouTube → đăng ký thành công.

Cấu trúc code minh họa (JavaScript/Node.js)

1. Target Interface (giao diện mà YouTube mong đợi):

- VisaPayment: Có method pay(amount, cardNumber, expiry, cvv).

2. Adaptee (giao diện hiện có – Momo):

- MomoPayment: Có method payWithMomo(amount, phoneNumber, pin).

3. Adapter:

- MomoToVisaAdapter: Implement VisaPayment.
- Nhận đối tượng Momo trong constructor.
- Trong method pay():
 - Lấy dữ liệu từ Momo.
 - Chuyển đổi (ví dụ: quy đổi số dư Momo sang "thẻ ảo Visa", tạo cardNumber giả lập...).
 - Gọi momo.payWithMomo() để trừ tiền thật.
 - Trả về kết quả như thanh toán Visa thành công.

Cách sử dụng

```
const momo = new MomoPayment('0123456789', '123456'); // Tài khoản Momo
const adapter = new MomoToVisaAdapter(momo);           // Adapter chuyển đổi
```

```
// Đăng ký hội viên YouTube như dùng Visa
```

```
const youtube = new YouTubeMembership();
```

```
youtube.subscribe(30000, adapter); // Thanh toán 30.000đ/tháng bằng Momo qua Adapter
```

Kết quả: YouTube nghĩ bạn dùng Visa → thành công. Tiền thực tế trừ từ Momo.

Lợi ích của Adapter Pattern

- Tái sử dụng code cũ mà không cần sửa (legacy code, third-party library).
- Tích hợp nhanh các hệ thống không tương thích (payment gateway, API cũ/mới).
- Dễ mở rộng: Thêm adapter mới cho VNPAY, ZaloPay... mà không sửa code YouTube client.

- Rất phổ biến trong dự án lớn, team lớn, khi tích hợp nhiều dịch vụ bên thứ ba.

Kết luận

Adapter Pattern giống như "phiên dịch viên" trong phần mềm: giúp hai bên không tương thích vẫn hợp tác mượt mà. Trong ví dụ này, nó giải quyết vấn đề thực tế: đăng ký hội viên YouTube bằng Momo khi chỉ hỗ trợ Visa.

Builder Pattern: Code thanh lịch là khi phức tạp trở thành đơn giản đó là khi Dev lên LEVEL.

Giới thiệu: Builder Pattern – Khi phức tạp trở thành đơn giản, dev lên level

Chào mừng bạn quay trở lại với series Design Pattern theo Gang of Four (GoF). Video này là phần tiếp theo sau Prototype Pattern (video trước). Hôm nay chúng ta nói về Builder Pattern – một creational pattern rất phổ biến trong Java (khó tránh khỏi), nhưng trong JavaScript thì ít dùng hơn.

Builder có độ khó khoảng 4 sao (khá cao vì cần hiểu rõ cấu trúc), độ phổ biến khoảng 2 sao (không phải pattern hàng ngày trong JS, nhưng rất mạnh khi đối tượng phức tạp). Nếu bạn lập trình Java hoặc hệ thống lớn, đây là pattern "bắt buộc" phải biết.

Builder Pattern là gì?

Builder Pattern giúp xây dựng đối tượng phức tạp từng bước một (step-by-step), thay vì truyền hết tất cả tham số vào constructor (gây dài dòng và khó đọc).

Nó tách riêng phần xây dựng (builder) khỏi phần đối tượng cuối cùng (product), cho phép:

- Tạo nhiều biến thể khác nhau của cùng một đối tượng.
- Chỉ set những thuộc tính cần thiết, bỏ qua optional.
- Code sạch sẽ, dễ đọc, dễ mở rộng.

Giải thích khoa học: "Tách construction khỏi representation, cho phép xây dựng từng phần của object phức tạp."

Giải thích đơn giản (theo phong cách kênh): Khi bạn cần tạo object "nặng" với hàng tá thuộc tính (nhiều optional), đừng dùng constructor dài ngoằng – dùng Builder để "xây nhà" từng bước: cửa sổ, cửa chính, phòng ngủ... rồi mới hoàn thiện.

Ví dụ thực tế trong cuộc sống: Đặt món ở nhà hàng 5 sao

Hãy tưởng tượng bạn vào nhà hàng "Tip JavaScript" 5 sao. Bạn muốn đặt một bữa ăn đầy đủ:

- Món khai vị, món chính, tráng miệng.
- Số lượng mỗi món: 1, 2, 3...
- Thêm yêu cầu đặc biệt: ít cay, không hành, thêm salad...

Nếu dùng constructor thông thường: `new Order(monKhaiVi, monChinh, trangMieng, soLuong1, soLuong2, ...)` → dài kinh khủng, dễ nhầm.

Builder giúp:

- Bắt đầu: `new OrderBuilder()`
- Thêm từng bước: `.withMonChinh("Phở bò") .withSoLuong(2) .withMonKhaiVi("Salad") .withGhiChu("Ít cay")`
- Cuối cùng: `.build()` → nhận object Order hoàn chỉnh.

Code đẹp, dễ tùy chỉnh biến thể (bữa ăn chay, bữa ăn gia đình...).

Ví dụ code minh họa: Tạo cầu thủ FIFA Online (Messi & Ronaldo)

Dùng lại ví dụ FIFA Online từ video Prototype trước để so sánh.

Class `FifaOnlinePlayer`: Đại diện cầu thủ với thuộc tính: tên, tuổi, quốc tịch, vị trí, đội bóng, thống kê (bàn thắng, kiến tạo...).

Thay vì constructor dài: `new FifaOnlinePlayer("CR7", 39, "Portugal", "ST", "Al Nassr", {goals: 900, assists: 250})`

Dùng Builder:

```
const cr7 = new FifaOnlinePlayerBuilder()
.withName("CR7")
.withAge(39)
.withNationality("Portugal")
.withPosition("ST")
.withTeam("Al Nassr")
.withStats({ goals: 900, assists: 250 })
.build();
console.log(cr7); // Object cầu thủ hoàn chỉnh
```

// Tạo Messi biến thể khác

```
const messi = new FifaOnlinePlayerBuilder()
.withName("M10")
.withAge(38)
.withNationality("Argentina")
.withPosition("RW")
.withTeam("Inter Miami")
.withStats({ goals: 850, assists: 380 })
.build();
```

Builder chia nhỏ: mỗi method `withXxx()` set một phần → cuối cùng `.build()` trả object. Dễ thêm optional (ví dụ chỉ set tên + tuổi nếu không cần thống kê chi tiết).

Ưu điểm của Builder Pattern

- Code thanh lịch, dễ đọc (fluent interface: chain method).
- Dễ mở rộng: Thêm thuộc tính mới chỉ cần thêm method `withNewField()`.

- Kiểm soát tốt: Set giá trị bắt buộc hoặc validation từng bước.
- Tạo nhiều biến thể dễ dàng mà không duplicate code.

Nhược điểm

- Code builder dài hơn (nhiều method).
- Khi thêm thuộc tính mới, phải cập nhật builder → phức tạp hơn Prototype một chút.

So sánh Builder vs Prototype (từ video trước)

- Prototype: Sao chép (clone) object sẵn có để tạo biến thể nhỏ (chỉ thay đổi vài thuộc tính). Phù hợp khi object cơ bản ổn định, giống nhau nhiều (ví dụ: chung cư – các căn hộ giống hệt, chỉ thay số tầng, màu sơn). Dễ clone nhanh.
- Builder: Xây dựng từ đầu, tỉ mỉ từng chi tiết. Phù hợp khi object phức tạp, mỗi instance khác biệt lớn (ví dụ: nhà phố – mỗi nhà bố trí riêng: vị trí cửa sổ, bếp, phòng ngủ khác nhau).

Ví dụ FIFA:

- Prototype: Clone Ronaldo cơ bản rồi chỉnh stats → nhanh nếu nhiều cầu thủ giống nhau.
- Builder: Xây từng thuộc tính → linh hoạt hơn nếu cầu thủ có cấu trúc khác biệt (ví dụ Ronaldo ST, Messi RW với stats riêng).

Khi nào dùng Builder?

- Object có nhiều thuộc tính (nhiều optional).
- Constructor telescoping (constructor nhiều overload) gây rối.
- Muốn code fluent, dễ đọc (như Lombok @Builder trong Java).
- Cần validation từng bước hoặc immutable object.